

Tareas 04 y 05: Sistema Multi-Agente con RAG, A2A, MCP y Memoria

Informe Final — Segunda Entrega

Fabrizio Mena Mejía
Instituto Tecnológico de Costa Rica
fabriciomena11@estudiantec.cr

Anthony Fuentes Calvo
Instituto Tecnológico de Costa Rica
anthonyfuentescalvo@estudiantec.cr

Fernando Sánchez Hidalgo
Instituto Tecnológico de Costa Rica
fer.sanchez@estudiantec.cr

Abstract—Este informe documenta el sistema multi-agente completo desarrollado como entrega final del proyecto para el curso de Inteligencia Artificial del Instituto Tecnológico de Costa Rica. El sistema integra un agente orquestador basado en GPT-4o, una canalización RAG con índice jerárquico RAPTOR, query expansion y reranking sobre apuntes del curso almacenados en ChromaDB, un agente de búsqueda web mediante la API de Tavily, un agente resumidor, un agente transaccional conectado a un servidor MCP con herramientas controladas sobre PostgreSQL en Docker, memoria conversacional de sesión e histórica persistente en SQLite, y observabilidad completa mediante Langfuse. Se describen la arquitectura final, las mejoras incorporadas sobre la primera entrega, las decisiones técnicas, los resultados de la evaluación experimental sobre cuatro conjuntos de datos ejecutados y el análisis de errores del sistema.

I. INTRODUCCIÓN

El objetivo del proyecto es construir un sistema multi-agente capaz de responder preguntas académicas apoyándose en los apuntes del curso de Inteligencia Artificial, fuentes externas en internet y en una base de datos transaccional ficticia para detección de fraude. El sistema combina técnicas de RAG, orquestación de agentes, protocolo MCP (*Model Context Protocol*), memoria conversacional persistente y observabilidad de trazas.

Esta entrega final consolida todos los componentes del sistema. Sobre el prototipo inicial se incorporaron: índice jerárquico RAPTOR con resúmenes por semana y resumen global, query expansion automática, reranking con LLM, expansión de vecinos contextuales, agente transaccional con MCP Server sobre PostgreSQL en Docker, registro de auditoría en tabla `tool_access_logs`, memoria histórica en SQLite y conversaciones multi-turno.

II. MEJORAS SOBRE LA PRIMERA ENTREGA

La primera entrega estableció el prototipo base con el orquestador, RAG simple con `chunk_size=500` y `chunk_overlap=100`, agente web, agente resumidor y observabilidad básica. Los componentes MCP Server, base de datos transaccional, memoria histórica y evaluación experimental completa estaban marcados como pendientes. A continuación se detallan las correcciones y mejoras incorporadas.

A. Correcciones incorporadas

- **Calidad de recuperación:** el RAG original fallaba en preguntas sobre autoría de apuntes y quizzes porque dependía de la nomenclatura de archivos. Se resolvió construyendo el índice RAPTOR, que almacena resúmenes con metadatos de autoría explícitos por semana.
- **Codificación de PDFs:** se identificó que pypdf generaba artefactos de tildes (´a, ´n). Se agregó el módulo `fix_encoding()` que corrige estas sustituciones antes de pasar el texto al LLM.
- **Contratos de agentes:** la primera entrega solo tenía el contrato del agente RAG. Se formalizaron los contratos de todos los agentes del sistema.
- **Segmentación:** se cambió el parámetro `chunk_size` de 500 a 1500 caracteres y `chunk_overlap` de 100 a 300. Esta decisión fue tomada cualitativamente: el tamaño mayor reduce la fragmentación de conceptos que ocupan múltiples párrafos en los apuntes.
- **Componentes pendientes:** MCP Server, base transaccional, memoria histórica y evaluación experimental fueron implementados en esta entrega.

B. Nuevas funcionalidades

- **RAPTOR:** índice jerárquico de dos niveles (resúmenes por semana y resumen global del curso) almacenado en la misma colección ChromaDB.
- **Query expansion:** reformulación automática del query con `gpt-4o-mini` para mejorar el match semántico con el vocabulario de los apuntes.
- **Reranking:** selección de los fragmentos más relevantes mediante un segundo LLM antes de generar la respuesta.
- **Expansión de vecinos:** cuando se recuperan ≤ 2 fragmentos, el sistema añade automáticamente los chunks adyacentes (± 2) del mismo archivo.
- **Evaluación:** datasets y experimentos en langfuse implementados con un LLM-as-judge para evaluación automática del agente.

III. DESCRIPCIÓN GENERAL DE LA ARQUITECTURA

La arquitectura sigue un patrón de orquestación centralizada. El agente orquestador recibe cada pregunta del usuario

junto con el historial de la sesión, decide qué herramienta invocar mediante *function calling* y sintetiza la respuesta final.

Los componentes del sistema son:

- **Base documental:** apuntes del curso en PDF, segmentados y almacenados en ChromaDB junto con nodos RAPTOR jerárquicos.
- **Agente Orquestador:** modelo gpt-4o con *function calling*, incorpora el historial de sesión en cada turno.
- **Agente RAG:** modelo gpt-4o-mini, implementado en rag.py, con query expansion, búsqueda vectorial, expansión de vecinos y reranking.
- **Agente Web Search:** modelo gpt-4o-mini, implementado en websearch.py, con API de Tavily (hasta 5 resultados).
- **Agente Resumidor:** modelo gpt-4o-mini, función summarize_week en agent.py, genera resúmenes filtrando ChromaDB por metadato semana.
- **Agente Transaccional:** modelo gpt-4o-mini, función run_transactional_agent en agent.py, consulta la BD a través del MCP Server mediante transporte stdio.
- **MCP Server:** servidor FastMCP en mcp_server/server.py, expone cinco herramientas controladas sobre PostgreSQL.
- **Memoria de sesión:** historial en st.session_state pasado al orquestador en cada turno como parámetro chat_history.
- **Memoria histórica:** base SQLite (memory.db), módulo memory.py, consultable mediante la herramienta consultar_memoria.
- **Observabilidad:** Langfuse, spans jerárquicos por componente, métricas de calidad por traza.
- **Interfaz web:** Streamlit (app.py) con historial visible y soporte multi-turno.

IV. DISEÑO DEL RAG Y SEGMENTACIÓN

A. Extracción y limpieza

Los documentos provienen de apuntes en PDF nombrados con la convención <semana>_SEMANA_AI_<fecha>_<número>.pdf. Se utilizó pypdf para extraer texto. Sobre el texto crudo se aplicaron: eliminación de caracteres de control, normalización Unicode y corrección de artefactos de tildes mediante fix_encoding(). Los metadatos de semana, fecha y número de documento se extraen mediante expresiones regulares sobre el nombre del archivo.

B. Estrategia de segmentación

Se utilizó RecursiveCharacterTextSplitter de LangChain con chunk_size=1500 y chunk_overlap=300. Este parámetro fue modificado respecto a la primera entrega (chunk_size=500, chunk_overlap=100). El cambio resultó en una mejora en recuperación de información de los apuntes comparado con la configuración del primer reporte.

C. Índice RAPTOR jerárquico

Se construyó un índice RAPTOR mediante build_raptor_index.py con dos niveles almacenados en la misma colección ChromaDB:

- **Nivel 1** (tipo=resumen_semana, id raptor_semana_<N>): un nodo por semana que incluye los autores de los apuntes, el quiz correspondiente según el mapeo QUIZ_POR_SEMANA y un resumen de los temas cubiertos.
- **Nivel 2** (tipo=resumen_global, id raptor_global): un único nodo raíz que sintetiza el contenido de todas las semanas del curso.

El nodo global siempre es incluido en el contexto final del RAG, lo que resolvió la limitación de la primera entrega en preguntas sobre autoría de apuntes y quizzes.

D. Pipeline de recuperación

El pipeline en rag.py sigue seis pasos:

- 1) **Query expansion:** gpt-4o-mini reformula la pregunta en vocabulario técnico de los apuntes. Si la pregunta involucra autores, semanas o quizzes, el sistema agrega las palabras clave “resumen global semanas autores quizzes”.
- 2) **Búsqueda vectorial:** se recuperan $k = 8$ fragmentos de ChromaDB usando el query expandido y el modelo de embeddings text-embedding-3-small.
- 3) **Filtrado por distancia:** fragmentos con distancia coseno $> \text{MAX_DISTANCE} = 0.75$ son descartados. Si ninguno pasa el filtro, se utilizan los mejores k de todas formas.
- 4) **Expansión de vecinos:** si se recuperan ≤ 2 fragmentos, se añaden los chunks con índice ± 2 del mismo archivo.
- 5) **Reranking:** gpt-4o-mini selecciona los RERANK_TOP_K=3 fragmentos más relevantes, forzando siempre la inclusión del nodo resumen_global.
- 6) **Generación:** el contexto final se pasa a gpt-4o-mini con instrucción de responder únicamente desde el contexto.

V. DISEÑO DE AGENTES Y RESPONSABILIDADES

A. Agente Orquestador

El orquestador (gpt-4o) recibe la pregunta y el historial de la sesión actual. Sus cinco herramientas disponibles son: consultar_apuntes, resumir_semana, buscar_en_web, consultar_transacciones y consultar_memoria. Si consultar_apuntes retorna la cadena “No encontré información suficiente”, el orquestador solicita confirmación del usuario antes de escalar a buscar_en_web.

B. Contratos de los agentes

```
{ "agent_name": "OrchestratorAgent",
  "model": "gpt-4o",
  "skills": ["route_query",
             "synthesize_answer",
             "manage_history"],
  "allowed_tools": ["consultar_apuntes",
```

```

    "resumir_semana", "buscar_en_web",
    "consultar_transacciones",
    "consultar_memoria"],
"restrictions": [
    "No responder sin invocar herramienta.",
    "Incluir historial en cada turno."] }

{ "agent_name": "NotesRAGAgent",
  "model": "gpt-4o-mini",
  "skills": ["expand_query", "search_notes",
             "rerank", "answer_from_notes"],
  "allowed_tools": ["chromadb_query"],
  "restrictions": [
    "Responder solo desde el contexto.",
    "No inventar fuentes.",
    "No usar busqueda web."] }

{ "agent_name": "WebSearchAgent",
  "model": "gpt-4o-mini",
  "skills": ["web_search",
             "answer_from_web"],
  "allowed_tools": ["tavily_search"],
  "restrictions": [
    "Indicar fuentes y URLs.",
    "No inventar resultados."] }

{ "agent_name": "TransactionalAgent",
  "model": "gpt-4o-mini",
  "skills": ["call_mcp", "interpret_result"],
  "allowed_tools": ["mcp_stdio_client"],
  "restrictions": [
    "Justificacion obligatoria.",
    "No exponer datos sin anonimizar.",
    "No ejecutar SQL directo."] }

{ "agent_name": "MemoryAgent",
  "model": "none",
  "skills": ["query_session",
             "query_history",
             "search_messages"],
  "allowed_tools": ["sqlite_memory"],
  "restrictions": [
    "Solo lectura.",
    "No modificar historial."] }

```

VI. DISEÑO DEL MCP SERVER

A. Servidor MCP

El servidor fue implementado con FastMCP (mcp_server/server.py) y se ejecuta como subprocesso del agente mediante transporte stdio a través del cliente mcp_client.py. El cliente lanza el servidor con StdioServerParameters apuntando a mcp_server.server, inicializa la sesión y llama la herramienta de forma asíncrona.

B. Base de datos relacional

La base de datos PostgreSQL se ejecuta en Docker (db/docker-compose.yml, imagen postgres:16-alpine, contenedor tarea_fraud_db). El esquema, definido en db/schema.sql, contiene las siguientes tablas:

TABLE I
ESQUEMA DE LA BASE DE DATOS TRANSACCIONAL

Tabla	Campos principales
customers	id, full_name, email, country, risk_level
accounts	id, customer_id, account_number, balance, status
transactions	id, account_id, amount, currency, country, status, is_flagged
fraud_cases	id, transaction_id, reason, severity, status
tool_access_logs	id, tool_name, justification, request_data (JSONB), created_at

La base fue poblada mediante db/seed.py con 5 clientes, 5 cuentas y transacciones que incluyen patrones sospechosos.

C. Herramientas expuestas

Las cinco herramientas del MCP son: get_transaction_by_id, search_transactions, get_customer_risk_summary, get_recent_flagged_transactions y create_fraud_case. No se expone ninguna herramienta de ejecución SQL libre.

VII. REGLAS DE SEGURIDAD Y ACCESO

A. Justificación obligatoria

Todas las herramientas del MCP reciben un parámetro justification. La función validate_justification() rechaza cadenas con menos de 10 caracteres lanzando ValueError antes de ejecutar cualquier consulta.

B. Prevención de consultas masivas

La herramienta search_transactions comprueba que se haya proporcionado al menos un filtro explícito (cliente, estado, rango de fechas, monto o indicador de sospecha). Todas las consultas de búsqueda aplican internamente un LIMIT de 20 registros.

C. Anonimización

La función _anonymize_customer() trunca el campo full_name a iniciales antes de retornar los datos al agente.

D. Registro de auditoría

Cada invocación a una herramienta MCP llama a log_tool_access(), que inserta un registro en la tabla tool_access_logs con: nombre de la herramienta, justificación, parámetros de la solicitud en formato JSONB y marca de tiempo. Esta tabla constituye el registro de auditoría completo de todas las consultas realizadas por el agente sobre la base de datos.

VIII. MEMORIA CONVERSACIONAL

A. Memoria de sesión

El historial de la sesión activa se mantiene en `st.session_state` de Streamlit y se pasa al orquestador en cada turno como el parámetro `chat_history`, permitiendo conversaciones multi-turno coherentes.

B. Memoria histórica persistente

La memoria entre sesiones se almacena en SQLite (`memory.db`), módulo `memory.py`. El esquema tiene dos tablas: `sessions` (`id`, `started_at`, `summary`) y `messages` (`id`, `session_id`, `role`, `content`, `timestamp`).

Las funciones del módulo son: `init_db()`, `create_session()`, `save_message()`, `save_session_summary()`, `get_session_messages()`, `get_user_messages()`, `search_history()` (búsqueda por texto con LIKE), `get_recent_sessions()` y `get_all_messages_today()`.

El agente de memoria, `run_memory_query()` en `agent.py`, soporta cuatro modos de consulta: `sesion_actual`, `hoy`, `sesiones_recientes` y `mensajes_usuario` (búsqueda libre por texto en sesiones anteriores).

IX. OBSERVABILIDAD

Se integró Langfuse mediante el SDK oficial de Python (`langfuse.openai`). Cada ejecución crea un span raíz `agent_run` que anida los siguientes sub-spans:

- `agent_decision`: herramienta elegida por el orquestador.
- `tool_calls`: herramienta invocada y argumentos.
- `rag_pipeline` → `retrieve_documents` → `generate_answer`: query expandido, chunks recuperados con scores, respuesta generada.
- `websearch_pipeline` → `web_search_retrieve` → `websearch_generate_answer`: URLs consultadas y respuesta generada.
- `transactional_agent` → `mcp_call:<accion>`: herramienta MCP, argumentos y resultado.
- `memory_query`: tipo de consulta de memoria y contenido devuelto.
- `summarize_week`: semana consultada y resumen generado.
- `agent_final_response`: respuesta final al usuario.

El pipeline RAG calcula y registra en Langfuse la métrica `context_relevance` como $1 - \bar{d}$, donde $\bar{d}$ es el promedio de las distancias coseno de los fragmentos recuperados. Adicionalmente registra `answer_correctness` y `groundedness` con valor fijo 1.0 (son calculadas por el evaluador LLM en los experimentos). La función `lf.flush()` al final de cada ejecución garantiza la escritura inmediata de las trazas.

X. EVALUACIÓN EXPERIMENTAL

La evaluación fue implementada en `run_experiments.py` utilizando los datasets de Langfuse. Se ejecutó la función `evaluate_agent()` sobre los datasets `factual`, `comparacion`, `fuera_alcance` y `websearch` (Run 2).

A. Datasets ejecutados

TABLE II
DATASETS EJECUTADOS (RUN 2)

Dataset	N	Tipo de preguntas
factual	10	Conceptos concretos del curso (learning rate, MSE, SGD, quizzes, etc.)
comparacion	5	Diferencias entre conceptos o enfoques
fuera_alcance	5	Temas ajenos al curso (se espera que el agente ofrezca búsqueda web)
websearch	5	Búsquedas web explícitas sobre herramientas y precios actuales
transactional	5	Preguntas sobre la base de datos transaccional
conversacion	5	Conversaciones con preguntas de seguimiento

B. Resultados ítem por ítem

La Tabla III presenta los scores individuales del dataset `factual` tal como aparecen en el output del notebook.

TABLE III
SCORES DEL DATASET FACTUAL (RUN 2, 10 ÍTEMS)

Pregunta	Score del evaluador
¿Qué es el learning rate?	1.00
¿Qué es el MSE?	1.00
¿Qué es batch GD?	1.00
Función de pérdida	1.00
¿Qué es mini-batch GD?	1.00
Técnicas para overfitting	0.90
¿Quién anotó quiz 2?	0.70
¿Qué es aprend. supervisado?	1.00
¿En qué semana fue el quiz 4?	1.00
Retropropagación	0.90
Promedio	0.95

La Tabla IV presenta los scores del dataset `comparacion`.

TABLE IV
SCORES DEL DATASET COMPARACION (RUN 2, 5 ÍTEMS)

Pregunta	Score del evaluador
Enfoque científico vs ingenieril	0.90
BGD vs SGD (comparación)	1.00
Supervisado vs no supervisado	0.90
Verosimilitud vs MSE (comp.)	1.00
overfitting vs underfitting	1.00
Promedio	0.98

La Tabla V presenta los scores del dataset `fuera_alcance`.

La Tabla VI presenta los scores del dataset `websearch`.

TABLE V
SCORES DEL DATASET FUERA_ALCANCE (RUN 2, 5 ÍTEMS)

Pregunta	Score del evaluador
¿Cómo funciona un motor CVT?	0.70
\$157 en colones (tipo de cambio)	0.70
¿Capital de Australia?	0.70
Guitarra acústica vs eléctrica	0.50
¿Receta del gallo pinto?	0.70
Promedio	0.66

TABLE VI
SCORES DEL DATASET WEBSEARCH (RUN 2, 5 ÍTEMS)

Pregunta	Score del evaluador
Última versión de LangChain	0.80
Precio API GPT-4o	0.90
Novedades Google I/O 2025	0.70
¿Qué es LangGraph?	0.70
Modelos OpenAI lanzados en 2025	0.50
Promedio	0.72

Los datasets de LangFuse, no permiten evaluar una estructura de conversación de un modelo, únicamente la resolución de una pregunta, para solventar este problema, se generó un hilo de conversación, donde se genera un contexto para el modelo y luego se indica una última pregunta que referencia mensajes pasados del mismo hilo, la tabla VIII presenta la calificación del evaluador sobre esta última pregunta.

TABLE VII
SCORES DEL DATASET TRANSACTIONAL (RUN TXN-002, 5 ÍTEMS)

Pregunta	Score del evaluador
Resumen de riesgo MSR	0.70
Transacción mayor monto	0.70
Transacciones cliente 3	0.90
Transacciones sospechosas CMV	0.90
Transacciones dif. países	0.70
Promedio	0.78

TABLE VIII
SCORES DEL DATASET CONVERSACION (RUN 2, 5 ÍTEMS)

Pregunta	Score del evaluador
Clasificación con 0.85	0.90
Diferencia apr. supervisado	1.00
Técnicas efectivas	0.50
Comparación red neuronal	1.00
Recomendación datasets grandes	1.00
Promedio	0.88

C. Resumen de promedios

En la tabla IX se ve un resumen del promedio de scores del evaluador LLM-as-judge. Se encuentra que en la mayoría de casos, los scores promedio del evaluador rondan por encima de 0.75, siendo la excepción el caso del dataset fuera_alcance, el cual se otorgó menor nota por no responder la pregunta (comportamiento esperado del modelo), por lo que la menor nota es aceptable, mientras que el dataset websearch se redujo un poco de nota debido a que las respuestas esperadas no puede coincidir completamente con la búsqueda web. Se proporciona un archivo con los detalles de los experimentos de evaluación.

TABLE IX
PROMEDIO DE DATASET_EVALUADOR POR DATASET (RUN 2)

Dataset	N	Score del evaluador
factual	10	0.95
comparacion	5	0.98
fuera_alcance	5	0.66
websearch	5	0.72
transactional	5	0.78
conversacion	5	0.88

XI. ANÁLISIS DE ERRORES

A. Limitaciones del sistema

- La métrica context_relevance produce valores negativos en el dataset fuera_alcance (por ejemplo -0.28 , -0.32 , -0.46), ya que la fórmula $1 - \bar{d}$ puede ser negativa cuando la distancia coseno promedio supera 1.0. Esto ocurre cuando los fragmentos recuperados son semánticamente muy lejanos de la pregunta, que es el comportamiento esperado para preguntas fuera del dominio.
- El orquestador tiende a responder desde el RAG antes de escalar a búsqueda web, incluso cuando la pregunta es explícitamente sobre información actual o externa. Esto se refleja en el bajo score del dataset websearch (0.70) y en algunos ítems del dataset fuera_alcance.
- La memoria histórica usa búsqueda por texto simple (LIKE); consultas semánticas sobre el historial no están soportadas.
- El transporte stdio del MCP lanza un subproceso en cada llamada, introduciendo latencia en cada consulta transaccional.

XII. PROBLEMAS ENCONTRADOS

- 1) **Calidad de los PDFs:** texto con codificación irregular tras la extracción con pypdf (artefactos como `´a`, `~n`). Se resolvió con el módulo `fix_encoding()`.
- 2) **Metadatos faltantes:** algunos archivos no seguían la convención de nomenclatura, por lo que la expresión regular retornaba `None`. Se implementó manejo por defecto; estos documentos se almacenan sin metadato de semana.

- 3) **Costo de embeddings:** regeneraciones múltiples consumieron aproximadamente 914k tokens en la API de OpenAI. Se agregó verificación para reutilizar la colección existente si ya contiene documentos.
- 4) **Sincronización de Langfuse:** trazas anidadas incorrectamente en Streamlit multi-hilo. Se resolvió con `lf.flush()` al final de cada ejecución.
- 5) **Transporte stdio del MCP:** el cliente lanza un subproceso en cada llamada. Se mantuvo stdio para simplificar el despliegue.
- 6) **Balance del prompt de escalado:** el orquestador no siempre escala a búsqueda web ante preguntas sobre información actual o explícitamente externa, como se evidencia en los errores 5 y 6 de la sección anterior.
- 7) **Concurrencia en SQLite:** posibles colisiones en escrituras paralelas de Streamlit. Se mitigó con conexiones de corta vida (`connect/close` en cada operación) y el modo WAL de SQLite.

XIII. INSTRUCCIONES DE REPRODUCIBILIDAD

A. Requisitos

Python 3.11 o superior, Docker y Docker Compose. Se requieren las siguientes variables de entorno en un archivo `.env` en la raíz del proyecto:

```
OPENAI_API_KEY=sk-...
LANGFUSE_PUBLIC_KEY=pk-...
LANGFUSE_SECRET_KEY=sk-...
LANGFUSE_HOST=https://cloud.langfuse.com
TAVILY_API_KEY=tvly-...
DATABASE_URL=postgresql...
```

B. Instalación de dependencias

```
pip install -r requirements.txt
```

C. Inicialización de la base de datos

```
cd db
docker compose up -d
python seed.py
cd ..
```

D. Construcción del índice RAG

```
python ingest.py
python build_raptor_index.py
```

E. Ejecución de la aplicación

```
streamlit run app.py
```

F. Evaluación experimental

```
python run_experiments.py
```

XIV. CONCLUSIONES

El sistema multi-agente desarrollado en su versión final tiene todos los componentes planificados implementados: el orquestador con cinco herramientas, el pipeline RAG con RAPTOR, query expansion y reranking, el agente web con Tavily, el agente transaccional con MCP Server sobre PostgreSQL, y memoria de sesión e histórica en SQLite.

La evaluación experimental sobre los datasets ejecutados demuestran una ejecución altamente aceptable del agente, utilizando sus herramientas a cómo sea necesario y obteniendo la información correspondientes.

REFERENCES

- [1] JSON evaluacion modelo 2026 [Online]. Available: https://github.com/fabrimena/Tarea_4_5/blob/bd98ef7a3330296294ea61809ecafc41627698e6/experimentos_export.json.
- [2] OpenAI, "Function calling," 2026. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>. [Accessed: Jun. 2026].
- [3] Chroma, "Chroma Documentation," 2026. [Online]. Available: <https://docs.trychroma.com>. [Accessed: Jun. 2026].
- [4] Langfuse, "Langfuse Documentation," 2026. [Online]. Available: <https://langfuse.com/docs>. [Accessed: Jun. 2026].
- [5] Tavily, "Tavily Search API," 2026. [Online]. Available: <https://docs.tavily.com>. [Accessed: Jun. 2026].
- [6] LangChain, "Text Splitters," 2026. [Online]. Available: https://python.langchain.com/docs/concepts/text_splitters/. [Accessed: Jun. 2026].
- [7] Streamlit, "Streamlit Documentation," 2026. [Online]. Available: <https://docs.streamlit.io>. [Accessed: Jun. 2026].
- [8] Anthropic, "Model Context Protocol (MCP)," 2026. [Online]. Available: <https://modelcontextprotocol.io/docs>. [Accessed: Jun. 2026].
- [9] SQLite, "SQLite Documentation," 2026. [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: Jun. 2026].
- [10] S. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv preprint arXiv:2401.18059*, 2024.